

SD4 — Distributed Key-Value Store (Amazon SDE II / Google L4 bar, L5 extensions)

Oracle: Alex Xu ch. 6 + aggregator-confirmed. Variants: "design DynamoDB/Redis", "global KV store", "how does your cache scale past one box". JD tie-in (`../0.req/oracle_senior_ic3.pdf`): "relational databases, NoSQL systems, distributed storage" is named explicitly — this is the design that proves it. Tags: `[hot path]` · `[async]` · `[control plane]` . ? = likely follow-up with the answer to give.

Index

0. Time budget
 1. Why this is hard
 2. Crux
 3. Clarifying questions
 4. Functional requirements
 5. Non-functional requirements
 6. Back-of-envelope
 7. High-level design
 8. API
 9. Data model + storage engine
 10. Deep dives (4)
 11. Hygiene checklist
 12. Operational readiness
 13. Pop-up questions
 14. Trade-offs + defeaters
 15. Wrap-up script
 16. Memorization card
 17. References
-

\$0 Time budget

Section	Min
Clarify + crux	6
FR / NFR / BoE	6
HLD (ring, replicas, paths)	9
API + storage engine	5
Deep dives (2 of 4)	11
Tradeoffs + wrap	4
Slack	4

🧠 "I'll scope, then design the ring and the write path, and I'd like to go deep on quorum consistency and on the storage engine — steer me if you'd rather do failure handling."

§1 Why this is hard

naive: one Redis box → get/put, done

breaks at: dataset > RAM of one box; box dies → all keys gone; 1 box = 1 region latency

- Naive design → single node hash map (Redis/one Postgres table)
- First thing that breaks → capacity: 10 TB doesn't fit; so **partition** → who owns key K? → then durability: a partition's node dies → its keys are gone; so **replicate** → now replicas can disagree → partitioning + replication together force an answer to consistency — that's the whole problem
- Minimum primitives that fix it → consistent hashing (ownership that survives membership change) → quorum reads/writes (tunable consistency without a leader) → anti-entropy (replicas converge after failures)
- What the user wants → "put returns, and a later get sees it — fast, from anywhere, even during a node failure"
- Cost of being wrong → lost write (durability) is unforgivable; stale read (consistency) is negotiable — say which the interviewer's use case tolerates

🧠 "So this is a **partition-plus-replicate problem**, and every decision downstream is 'what happens to reads and writes while membership or replicas disagree.'"

? *Why not just Redis Cluster / DynamoDB?* → "That's what I'd deploy. The interview design **is** DynamoDB's paper: consistent hashing, quorums, hinted handoff, merkle anti-entropy. I'll design it from parts so we can argue the tradeoffs."

§2 Crux

- Hard part: **agreeing who owns a key and which value is newest, with no global coordinator**
- Critical success points → deterministic placement: hash ring + virtual nodes; membership via gossip → write survives node failure: W replicas ack; hinted handoff for the missing → read sees the newest acked write when configured to: $R + W > N$ → conflict story: last-write-wins timestamps vs vector clocks — pick and defend → node storage engine that makes random writes cheap: LSM tree
- Business metric → system metric → "checkout never loses the cart" → `write_success_rate` at $W=2$ + `hinted_handoff_backlog`

§3 Clarifying questions (with assumed answers)

Question	Assumed
Value size / shape?	opaque blob ≤ 400 KB (DynamoDB's item cap as anchor)
Scale?	10 TB, 100k reads/s, 10k writes/s
Consistency?	tunable per request; default eventual, option strong
Latency target?	p99 read < 10 ms in-region
Durability?	no acked write lost on ≤ 1 node failure ($N=3, W=2$)

Question	Assumed
Multi-region?	single region v1; mention cross-region async
Delete semantics?	yes — tombstones needed
Range scans / transactions	no (that's a different store) — non-goal

§4 Functional requirements

- Core → `put(key, value)`, `get(key)`, `delete(key)`, TTL optional → per-request consistency hint (`R=1` fast vs `R=quorum` fresh)
- Extended (mention, defer) → conditional put (CAS), batch ops, secondary indexes, change streams
- Non-goals → SQL, joins, multi-key transactions, range queries in v1

§5 Non-functional requirements

Requirement	Target	Why	Enforced by
Read latency	p99 < 10 ms	hot-path lookups	memtable + block cache + bloom filters
Write latency	p99 < 15 ms	user-facing puts	WAL append + memtable, no in-place update
Availability	99.99% for R=1/W=1 paths	serving tier dependency	no leader; any replica serves; sloppy quorum
Durability	acked write survives 1 node + 1 FD	trust	N=3 across FDs, $W \geq 2$, WAL fsync
Consistency	R+W>N option; default eventual	per-use-case	coordinator quorum logic
Scale	add node → ~1/M data moves	ops sanity	consistent hashing, vnodes
Security	per-tenant namespace + authz	multi-tenant (JD)	gateway authz; encryption at rest

L5: state the consistency SLO like an SLA: "R=W=2, N=3 gives read-your-writes for a single key **from the same client session**; cross-client ordering is not promised — if the interviewer's use case needs it, this is the wrong store, use a leader-based one (Raft group per shard)."

§6 Back-of-envelope

- 10 TB / N=3 → 30 TB raw; 30 nodes × 1 TB useful (NVMe, leave headroom) — say 40 nodes
- 100k reads/s ÷ 40 → 2.5k/node; 10k writes/s ÷ 40 → 250/node — comfortable for LSM
- Ring: 40 physical × 128 vnodes = 5,120 tokens → ±2% load imbalance (vnode count is why)
- Memory: bloom filter ~10 bits/key; 10 B keys → ~12 GB filters fleet-wide — cheap vs disk reads
- So what: node count is driven by **storage**, not CPU → value compression pays directly

§7 High-level design

```

[hot path] Client → any node (coordinator for this request)
                | hash(key) → ring → preference list [A, B, C] (N=3, next distinct FDs)
put —————→ send to A,B,C; ack client at W acks; C down → D holds hint [async]
get —————→ read R replicas; newest version wins; stale replica → read repair [async]
[async]      anti-entropy: merkle tree diff between replicas per key-range, repair in background
[control plane] gossip: membership + health every 1 s; ring recomputed locally, no central master

```

- Every node is identical (no leader) — coordinator is just "whichever node the client hit"
- Walkthrough — put(k,v):
 1. Coordinator computes preference list [A,B,C]
 2. Sends (k, v, version) to all three, waits for W=2 acks → 200 to client
 3. C is down → D (next on ring) accepts a **hinted** copy tagged "for C" → replays when C returns
- Walkthrough — get(k) at R=2:
 1. Ask A, B → versions differ (B missed an update)
 2. Return newest to client; asynchronously write newest back to B (**read repair**)

§8 API

```

PUT /v1/ns/{namespace}/keys/{key}    body: value  headers: X-Consistency: W1|W2, If-Match: <version>
(CAS)
→ 200 {version}    409 on CAS mismatch
GET /v1/ns/{namespace}/keys/{key}    headers: X-Consistency: R1|R2
→ 200 value + ETag: version    404
DELETE /v1/ns/{namespace}/keys/{key} → 204    (writes tombstone, GC'd after grace)

```

- Version = per-key monotonic (timestamp_ms, node_id, counter) — also the LWW tiebreak
- Idempotency: PUT with same version key is naturally idempotent; retries safe

§9 Data model + storage engine

Per-node engine = LSM tree (why: random-write-heavy; B-tree would seek per write):

```

write → WAL append (fsync) → memtable (skiplist) → [full] flush to SSTable (sorted, immutable)
read  → memtable → bloom filter per SSTable → newest SSTable hit wins
background: compaction merges SSTables, drops overwritten versions + expired tombstones

```

```

-- logical record (what an SSTable row holds)
-- key          bytes    partition key, ring-hashed
-- version      (ts_ms, node_id, ctr)  LWW ordering
-- value        bytes    nullable – null + tombstone=true means deleted
-- ttl_expires_at ts      optional
-- tombstone     bool

-- control-plane metadata (small, strongly consistent store, e.g. etcd)
CREATE TABLE ring_membership (
  node_id      text PRIMARY KEY,
  tokens       bigint[] NOT NULL,          -- 128 vnode positions
  fd           text NOT NULL,              -- fault domain
  state        text NOT NULL,              -- JOINING | UP | LEAVING | DOWN
  updated_at   timestampz NOT NULL
);
CREATE TABLE namespaces (
  namespace    text PRIMARY KEY,
  tenant_id    uuid NOT NULL,
  n_replicas   int  NOT NULL DEFAULT 3,
  default_r    int  NOT NULL DEFAULT 1,
  default_w    int  NOT NULL DEFAULT 2,
  quota_bytes  bigint
);

```

? *Why is delete a tombstone?* → "A plain delete on one replica resurrects via anti-entropy from a replica that never saw it. The tombstone is a versioned write that says 'deleted'; it wins by version and is GC'd only after every replica has seen it (grace period > max repair time — Cassandra's `gc_grace_seconds`)." [\[1\]](#)

§10 Deep dives

10.1 Consistent hashing + virtual nodes (placement)

- Problem: `hash(key) % N` reshuffles nearly every key when N changes
- Design: ring of hash space; each node owns arcs; key → first node clockwise; adding a node moves only ~1/M of keys; vnodes smooth load and let heterogeneous nodes take proportional tokens

```

import bisect
import hashlib

def _h(b):
    return int.from_bytes(hashlib.md5(b).digest()[:8], "big")

class Ring:
    def __init__(self, vnodes=128):
        self.vnodes = vnodes
        self.tokens = []          # sorted [(token, node)]
        self.fd = {}              # node -> fault domain

    def add(self, node, fd):
        self.fd[node] = fd
        for i in range(self.vnodes):
            bisect.insort(self.tokens, (_h(f"{node}#{i}".encode()), node))

    def remove(self, node):
        self.tokens = [(t, n) for t, n in self.tokens if n != node]
        del self.fd[node]

    def preference_list(self, key, n=3):
        """First n distinct nodes clockwise, spread across fault domains."""
        i = bisect.bisect(self.tokens, (_h(key.encode()), ""))
        out, fds = [], set()
        for j in range(len(self.tokens)):
            _, node = self.tokens[(i + j) % len(self.tokens)]
            if node in out or (self.fd[node] in fds and len(fds) < len(set(self.fd.values()))):
                continue
            out.append(node); fds.add(self.fd[node])
            if len(out) == n:
                return out
        return out

# Time: O(log T) lookup, T = tokens; add/remove O(V log T)

```

```

public final class Ring {
    private final TreeMap<Long, String> tokens = new TreeMap<>();
    private final Map<String, String> fd = new HashMap<>();
    private final int vnodes;

    Ring(int vnodes) { this.vnodes = vnodes; }

    void add(String node, String faultDomain) {
        fd.put(node, faultDomain);
        for (int i = 0; i < vnodes; i++) tokens.put(hash(node + "#" + i), node);
    }

    List<String> preferenceList(String key, int n) {
        List<String> out = new ArrayList<>(n);
        Set<String> fds = new HashSet<>();
        SortedMap<Long, String> tail = tokens.tailMap(hash(key));
        Iterator<String> it = Stream.concat(tail.values().stream(),
tokens.values().stream()).iterator();
        while (it.hasNext() && out.size() < n) {
            String node = it.next();
            if (out.contains(node)) continue;
            if (fds.contains(fd.get(node)) && fds.size() < new HashSet<>(fd.values()).size()) continue;
            out.add(node); fds.add(fd.get(node));
        }
        return out;
    }

    private long hash(String s) { /* md5 or murmur3 first 8 bytes */ return s.hashCode(); } // real
impl: murmur3
}
// TreeMap.tailMap = the "clockwise walk"; O(log T) per lookup

```

? Why 128 vnodes? → "Load stddev shrinks $\sim 1/\sqrt{V}$. 128 gives $\sim \pm 2-5\%$; more vnodes = bigger membership metadata and slower repair streaming. Cassandra default moved 256 → 16 (with a smarter allocator) for that reason."

10.2 Quorum consistency: $R + W > N$ (data/consistency)

- $N=3$. Configs to recite: → $W=2, R=2$ → overlap guaranteed → read sees newest acked write (per key) → $W=1, R=1$ → fastest, both windows of staleness and loss (node dies before replicating) → $W=3$ → durable but any one slow/dead node blocks writes — availability inverted
- **Sloppy quorum** (availability mode): if C is down, write lands on D with a hint → W still met, but $R+W > N$ no longer overlaps the *home* set → a quorum read can miss it until handoff replays. Say this honestly — it's the Dynamo tradeoff
- Conflict resolution → LWW (timestamp): simple; concurrent writes silently drop one — fine for caches/carts-with-merge-at-app → Vector clocks: detect true concurrency, return siblings, client merges — correct, complex; Dynamo did it, DynamoDB dropped it for LWW. Pick LWW here, name the loss

```
def quorum_read(replicas, key, r):
    """Ask all, take first r; newest version wins; repair the stale ones."""
    responses = gather_first(r, [rep.get(key) for rep in replicas]) # (version, value)
    winner = max(responses, key=lambda x: x.version)
    for rep, resp in responses.items():
        if resp.version < winner.version:
            rep.put_async(key, winner.value, winner.version) # read repair
    return winner
```

? *Is $R+W > N$ linearizable?* → "No — it gives per-key read-your-acked-writes, but a read racing an in-flight write can see it on one replica and not another; without read repair + a second round you can even read newer-then-older. Linearizable needs a leader (Raft) or ABD-style two-phase reads. Say 'quorum ≠ linearizable' and you pass the follow-up."

10.3 Failure handling: gossip, hinted handoff, anti-entropy (resilience)

- Detection: gossip heartbeats; node marked suspect after missed beats (phi-accrual, not fixed timeout)
- Short outage → **hinted handoff**: neighbor stores writes tagged for the dead node; replays on return; cap hint storage (e.g., 3 h) or a long outage floods the neighbor
- Long outage / lost disk → **anti-entropy**: replicas exchange merkle trees per key range; only differing subtrees stream → repair cost $\propto$ divergence, not data size
- Permanent removal: operator `decommission` → ranges stream to new owners; ring metadata via the strongly-consistent store, applied via gossip
- Blast radius: preference list spans fault domains → one FD loss keeps 2/3 replicas (OCI: spread across FDs in an AD; cross-AD = the multi-AZ analog)

10.4 Hot keys + large values (flow control)

- Hot partition: one celebrity key → its 3 replicas melt → detect: per-key QPS top-k (count-min sketch) on coordinators → fix: request coalescing at coordinator; key-local cache (front tier); for write-hot, split key `k` → `k#1..k#8` shards, merge on read (app-visible — say so)
- Large values: 400 KB cap enforced at API; bigger → store in object storage, KV holds pointer (the "claim check" pattern)
- Backpressure: per-node admission control — shed $R=1$ reads first, protect writes (durability over freshness)

§11 Hygiene checklist

Concern	Default here
AuthN/Z	gateway OIDC; namespace ↔ tenant policy; node-to-node mTLS (JD: PKII!)
Idempotency	versioned puts; retries safe
Timeouts/retries	per-replica 50 ms timeout, hedge the 3rd replica after p95
Consistency	per-request R/W; document sloppy-quorum caveat
Storage	LSM + WAL fsync; compaction throttled to protect p99
Replication	$N=3$ across FDs; hinted handoff 3 h cap; merkle repair weekly + on-demand

Concern	Default here
Observability	per-node p99, compaction backlog, hint backlog, repair divergence, imbalance
Deploy	one node at a time per FD; drain = stop coordinating, keep serving replicas
Retention/PII	value encryption at rest per namespace key; tombstone GC grace 10 d
Multi-tenancy	namespace quotas (bytes + QPS); per-tenant admission control

§12 Operational readiness

- Capacity: disk 70% alarm (compaction needs headroom); add nodes before 80%
- Runbook → **p99 read spike** → block cache hit ratio down? compaction storm? → throttle compaction, check hot-key top-k → **hint backlog growing** → a node flapping → stop the flap (restart/replace), let handoff drain, verify with merkle diff → **imbalance > 10%** → vnode allocator or a fat tenant → rebalance tokens / split namespace
- Chaos tests: kill a node mid-write (assert no acked write lost); partition an FD 5 min (assert availability at R=1, repair after)
- Weekly: full merkle repair; report divergence trend (rising = something eats writes)

§13 Pop-up questions

- CAP?* → "Under partition choose A or C **per operation**: sloppy quorum W=1 is AP; R=W=quorum on the home set leans CP for that key. The knob is per-request — that's the design's point."
- Why no leader?* → "Leaderless = any node serves = 99.99% availability and no failover pause. Cost: no linearizability, conflict handling pushed to versions. Need transactions/ordering → Raft group per shard (different store)."
- Read-your-writes for a session?* → "Client sticks to the same coordinator + R≥W overlap; or session token carrying last-seen version, coordinator waits until a replica catches up."
- How does a new node join?* → "JOINING in metadata → streams its ranges from current owners → gossip flips it UP → ring updates locally everywhere; no global pause."
- Clock skew breaks LWW?* → "Yes — LWW with wall clocks can drop the *later* real write. Mitigate: hybrid logical clocks (HLC); or vector clocks if correctness > simplicity."
- Compaction strategies?* → "Size-tiered (write-optimized, space spikes) vs leveled (read-optimized, write amplification). Read-heavy → leveled. Name, don't derive."
- 10x?* → "400 nodes: gossip and merkle still fine; watch metadata store QPS and rebalance time; consider range-based partitioning if scans ever appear."

§14 Trade-offs

Decision	Chosen	Alternative	Flips when
Topology	leaderless + quorum	leader per shard (Raft)	need linearizable/CAS-heavy/transactions
Conflicts	LWW version	vector clocks + siblings	concurrent writes must both survive
Partitioning	hash ring + vnodes	range partitioning	range scans required
Engine	LSM	B-tree	read-mostly + in-place updates dominate
Availability	sloppy quorum + hints	strict quorum	consistency contractually required

- Defeaters → "R+W>N means strong consistency" — it's per-key overlap, not linearizability → "Consistent hashing needs a central ring server" — the ring is computed locally from gossiped membership → "Deletes can be plain removes" — resurrection via repair; tombstones or bust

§15 Wrap-up (2 min)

"Crux: ownership and freshness without a coordinator. Design: consistent-hash ring with vnodes and FD-aware preference lists, N=3 quorum replication with per-request R/W, hinted handoff + read repair + merkle anti-entropy for convergence, LSM per node. Default eventual, R+W>N when the caller needs read-your-writes, and I said out loud that quorum ≠ linearizable. MVP0: fixed membership, strict quorum, LWW. At 10×: more vnodes, leveled compaction, hot-key sharding."

§16 Memorization card

- Pitch: ring + vnodes → preference list; quorum W/R; hints + read repair + merkle; LSM node engine
- Why hard: partitioning and replication force a consistency answer with no global coordinator
- 5 numbers: N=3 W=2 R=2 · 128 vnodes · 400 KB value cap · 10 bits/key bloom · hint cap 3 h
- 3 failures: node down mid-write (hints) · replica divergence (merkle) · hot key (coalesce/shard)
- 3 defeaters: quorum=linearizable · central ring server · plain deletes
- Done when: draw ring + both walkthroughs in 4 min; write `preference_list` from memory; recite the R/W/N table

§17 References

- Dynamo paper (DeCandia et al., 2007) — the whole §10
- Alex Xu vol 1 ch. 6 — interview-shaped retelling
- Cassandra docs: tombstones/gc_grace, vnode allocator, phi-accrual detector
- Kleppmann DDIA ch. 5–6 — replication + partitioning, LWW pitfalls